UNIVERSIDADE CIDADE DE SÃO PAULO
ALGORITMOS E PENSAMENTO COMPUTACIONAL (APC)
Professor: Carlos Eduardo Weber

ENZO FREIRE DOS SANTOS – RGM: 47806303
EDUARDO VINICIUS RIBEIRO COSTA – RGM: 48693855
GIOVANNA RIBEIRO SOUZA – RGM: 47952822
JOÃO VICTOR DA COSTA SILVA – RGM: 4227238
PAULO HENRIQUE CAMILO DE PASCOA SOUZA – RGM: 47806745
KAIKE CARDOSO DA SILVA – RGM: 48401455
KEVYN MORAIS BARROS – RGM: 48328936

SISTEMA DE CAIXA ELETRÔNICO (ATM): DOCUMENTAÇÃO DE TESTES

São Paulo
2026

ENZO FREIRE DOS SANTOS – RGM: 47806303
EDUARDO VINICIUS RIBEIRO COSTA – RGM: 48693855
GIOVANNA RIBEIRO SOUZA – RGM: 47952822
JOÃO VICTOR DA COSTA SILVA – RGM: 4227238
PAULO HENRIQUE CAMILO DE PASCOA SOUZA – RGM: 47806745
KAIKE CARDOSO DA SILVA – RGM: 48401455
KEVYN MORAIS BARROS – RGM: 48328936

SISTEMA DE CAIXA ELETRÔNICO (ATM): DOCUMENTAÇÃO DE TESTES

Trabalho apresentado ao componente curricular Algoritmos e Pensamento Computacional (APC), como requisito parcial para avaliação da Atividade 2 Prática, da Universidade Cidade de São Paulo.

Professor: Carlos Eduardo Weber

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO 4

2 DESENVOLVIMENTO 4

2.1 Variáveis utilizadas 4

2.2 Lógica de distribuição das notas 4

2.3 Código-fonte 5

2.4 Testes e validação 6

2.5 Análise dos resultados 6

3 CONCLUSÃO 9

REFERÊNCIAS 9

1 INTRODUÇÃO

Este trabalho documenta o desenvolvimento e a validação de um sistema simplificado de caixa eletrônico (ATM) implementado em linguagem C, proposto na Atividade 2 Prática do componente Algoritmos e Pensamento Computacional (APC). O objetivo é aplicar conceitos de lógica de programação, estruturas condicionais e operadores aritméticos na resolução de um problema do cotidiano.

O programa solicita ao usuário o valor do saque, verifica se ele é válido e se não excede o saldo disponível e, em caso positivo, calcula a quantidade de cédulas e moedas de 100, 50, 20, 10, 5, 2 e 1 a serem entregues. Este relatório apresenta a metodologia, o código-fonte e as evidências dos testes realizados com os valores 50, 95, 123 e 1005, analisando os cenários válidos e inválidos.

O código-fonte e a versão editável do projeto estão disponíveis em repositório público (GRUPO, 2026).

2 DESENVOLVIMENTO

2.1 Variáveis utilizadas

Todas as variáveis são do tipo primitivo *int*, adequado a valores monetários inteiros. O Quadro 1 apresenta a identificação de cada uma.

Quadro 1 – Variáveis do programa

Variável	Tipo	Finalidade
SALDO	const int	Saldo disponível (fixado em R$ 1000)
valor	int	Valor que o usuário deseja sacar
resto	int	Parte do valor ainda não distribuída em notas
n100, n50, n20, n10, n5, n2, n1	int	Contadores de cada cédula ou moeda

Fonte: elaborado pelos autores (2026).

2.2 Lógica de distribuição das notas

O valor é primeiro validado: deve ser maior que zero e menor ou igual ao saldo. Como existem moedas de R$ 1 e R$ 2, todo valor inteiro positivo pode ser entregue, o que torna a verificação de múltiplos satisfeita por construção. Sendo válido, o cálculo é feito de forma encadeada: a divisão inteira (/) fornece a quantidade de cédulas de determinado valor e o operador módulo (%) fornece o resto, que segue para a cédula seguinte. Apenas as cédulas com quantidade maior que zero são exibidas.

Adotou-se, como premissa, saldo disponível fixo de R$ 1000, pois o roteiro da atividade não define o saldo (UNICID, 2026).

2.3 Código-fonte

O Quadro 2 reproduz o programa desenvolvido, compilado com o compilador GCC sem avisos.

Quadro 2 – Código-fonte em linguagem C

```
#include <stdio.h>

/* Sistema de Caixa Eletronico (ATM) - Atividade 2 Pratica (APC) */
int main(void)
{
    const int SALDO = 1000;   /* saldo disponivel (valor fixo para os testes) */
    int valor;                /* valor que o usuario deseja sacar */
    int resto;                /* valor ainda nao distribuido em notas */
    int n100, n50, n20, n10, n5, n2, n1; /* contadores de cada cedula/moeda */

    printf("=== CAIXA ELETRONICO ===\n");
    printf("Saldo disponivel: R$ %d\n", SALDO);
    printf("Digite o valor do saque: R$ ");
    if (scanf("%d", &valor) != 1) {          /* entrada nao numerica */
        printf("ERRO: entrada invalida.\n");
        return 1;
    }

    /* Validacoes: valor positivo e dentro do saldo.
       Como existem moedas de R$ 1 e R$ 2, qualquer inteiro positivo
       e multiplo de alguma combinacao de cedulas (2, 5, 10...). */
    if (valor <= 0) {
        printf("ERRO: valor invalido. Digite um valor inteiro maior que zero.\n");
    } else if (valor > SALDO) {
        printf("ERRO: saldo insuficiente. Saque maximo: R$ %d.\n", SALDO);
    } else {
        /* Divisao inteira (/) da a quantidade; modulo (%) da o resto */
        n100 = valor / 100;  resto = valor % 100;
        n50  = resto / 50;   resto = resto % 50;
        n20  = resto / 20;   resto = resto % 20;
        n10  = resto / 10;   resto = resto % 10;
        n5   = resto / 5;    resto = resto % 5;
        n2   = resto / 2;    resto = resto % 2;
        n1   = resto;

        printf("Saque de R$ %d autorizado. Distribuicao:\n", valor);
        /* Exibe apenas cedulas/moedas com quantidade maior que zero */
        if (n100 > 0) printf("  %d nota(s) de R$ 100\n", n100);
        if (n50  > 0) printf("  %d nota(s) de R$ 50\n",  n50);
        if (n20  > 0) printf("  %d nota(s) de R$ 20\n",  n20);
        if (n10  > 0) printf("  %d nota(s) de R$ 10\n",  n10);
        if (n5   > 0) printf("  %d nota(s) de R$ 5\n",   n5);
        if (n2   > 0) printf("  %d moeda(s) de R$ 2\n",  n2);
        if (n1   > 0) printf("  %d moeda(s) de R$ 1\n",  n1);
    }
    return 0;
}
```

Fonte: elaborado pelos autores (2026).

2.4 Testes e validação

O programa foi compilado e executado com os quatro valores exigidos e com um valor adicional (zero). A Tabela 1 resume os resultados, e as Figuras 1 a 9 apresentam as saídas obtidas no Portugol WebStudio e no programa em C.

Tabela 1 – Resumo dos testes executados

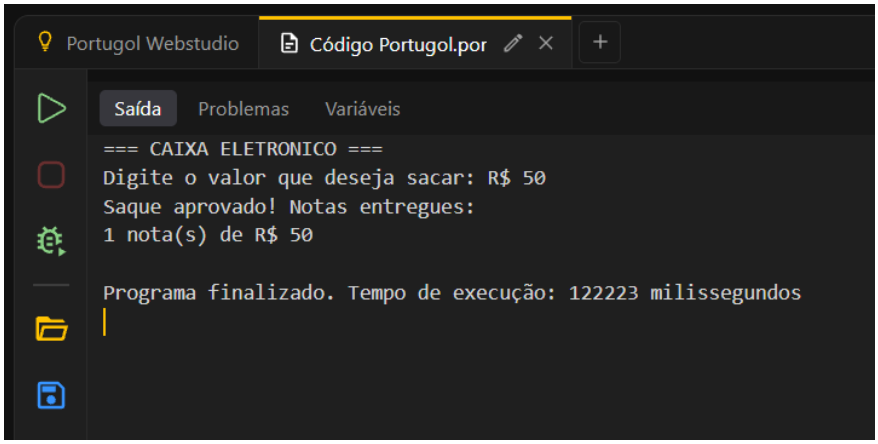
Entrada (R$)	Cenário	Resultado obtido
50	Válido	1 × R$ 50
95	Válido	1 × R$ 50; 2 × R$ 20; 1 × R$ 5
123	Válido	1 × R$ 100; 1 × R$ 20; 1 × R$ 2; 1 × R$ 1
1005	Inválido	Erro: saldo insuficiente
0	Inválido	Erro: valor inválido

Fonte: elaborado pelos autores (2026).

2.5 Análise dos resultados

Valor 50. Caso válido, pois é positivo e inferior ao saldo. A distribuição resulta em uma única nota de R$ 50.

Figura 1 – Execução do pseudocódigo no Portugol WebStudio com o valor 50



Fonte: elaborado pelos autores (2026).

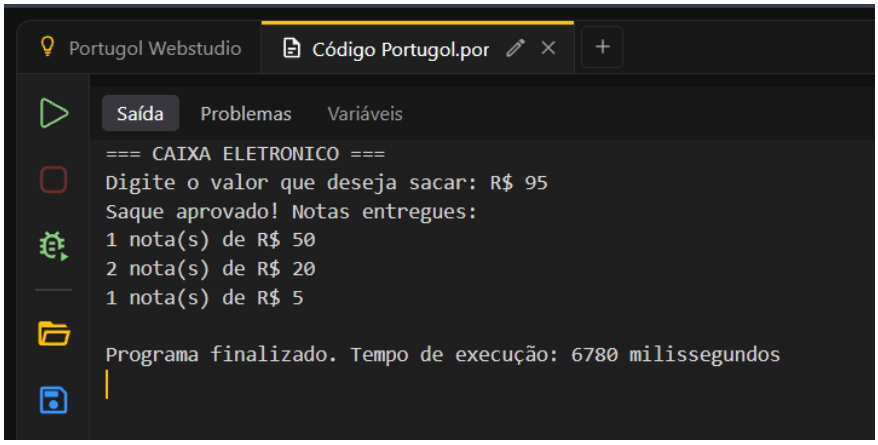
Figura 2 – Execução do programa em C com o valor 50

```
=== CAIXA ELETRONICO ===
Saldo disponivel: R$ 1000
Digite o valor do saque: R$ 50
Saque de R$ 50 autorizado. Distribuicao:
  1 nota(s) de R$ 50
```

Fonte: elaborado pelos autores (2026).

Valor 95. Caso válido. O valor não é múltiplo de 100 e a cadeia de divisões e restos gera 50 + 20 + 20 + 5, usando o menor número de cédulas.

Figura 3 – Execução do pseudocódigo no Portugol WebStudio com o valor 95



Fonte: elaborado pelos autores (2026).

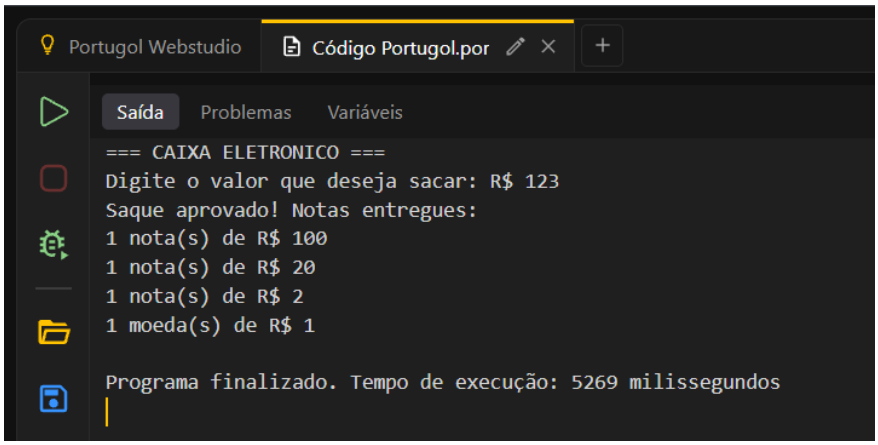
Figura 4 – Execução do programa em C com o valor 95

```
=== CAIXA ELETRONICO ===
Saldo disponivel: R$ 1000
Digite o valor do saque: R$ 95
Saque de R$ 95 autorizado. Distribuicao:
  1 nota(s) de R$ 50
  2 nota(s) de R$ 20
  1 nota(s) de R$ 5
```

Fonte: elaborado pelos autores (2026).

Valor 123. Caso válido. Mesmo sendo ímpar, é atendido graças à moeda de R$ 1: 100 + 20 + 2 + 1.

Figura 5 – Execução do pseudocódigo no Portugol WebStudio com o valor 123



Fonte: elaborado pelos autores (2026).

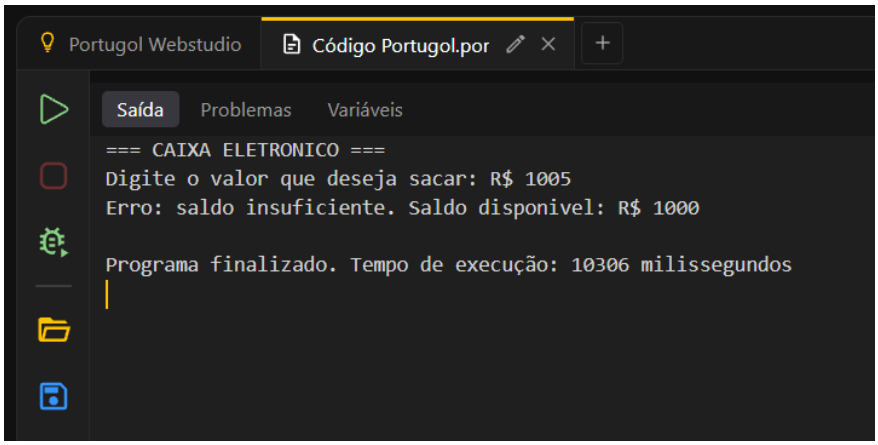
Figura 6 – Execução do programa em C com o valor 123

```
=== CAIXA ELETRONICO ===
Saldo disponivel: R$ 1000
Digite o valor do saque: R$ 123
Saque de R$ 123 autorizado. Distribuicao:
  1 nota(s) de R$ 100
  1 nota(s) de R$ 20
  1 moeda(s) de R$ 2
  1 moeda(s) de R$ 1
```

Fonte: elaborado pelos autores (2026).

Valor 1005. Caso inválido, pois excede o saldo de R$ 1000. O programa exibe mensagem de saldo insuficiente e não calcula notas.

Figura 7 – Execução do pseudocódigo no Portugol WebStudio com o valor 1005



Fonte: elaborado pelos autores (2026).

Figura 8 – Execução do programa em C com o valor 1005

```
=== CAIXA ELETRONICO ===
Saldo disponivel: R$ 1000
Digite o valor do saque: R$ 1005
ERRO: saldo insuficiente. Saque maximo: R$ 1000.
```

Fonte: elaborado pelos autores (2026).

Valor 0 (teste adicional). Caso inválido. Valores menores ou iguais a zero são rejeitados, evitando saques sem sentido.

Figura 9 – Execução do programa em C com o valor 0

```
=== CAIXA ELETRONICO ===
Saldo disponivel: R$ 1000
Digite o valor do saque: R$ 0
ERRO: valor invalido. Digite um valor inteiro maior que zero.
```

Fonte: elaborado pelos autores (2026).

3 CONCLUSÃO

Os resultados mostram que o programa atende aos requisitos da atividade: aceitou e distribuiu corretamente os valores 50, 95 e 123, e recusou o valor 1005 por saldo insuficiente, além de rejeitar entrada não positiva. O uso encadeado de divisão inteira e módulo mostrou-se eficiente, e a exibição restrita às cédulas com quantidade maior que zero tornou a saída clara. Como melhoria futura, sugere-se permitir saldo variável por conta e controle do estoque de cédulas do equipamento.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

GRUPO. **Sistema lógico de caixa eletrônico**. [S. l.]: GitHub, 2026. Disponível em: https://github.com/gioribeirro/Sistema-logico-de-caixa-eletronico. Acesso em: 17 out. 2026.

UNIVERSIDADE CIDADE DE SÃO PAULO. **Roteiro da Atividade 2 Prática**: sistema de caixa eletrônico (ATM). São Paulo: UNICID, 2026. Material do componente Algoritmos e Pensamento Computacional.